A GUIDE TO YOUR OWN CODEBASE

Understanding StudyHub

How the system actually works — traced end to end, in plain language, using the real code.

Written for

Tazanu Stanley — to understand, explain and defend the technical side of StudyHub.

Assumes you can program, but assumes nothing about web architecture, ORMs or payment rails.

CONTENTS

What is in here

This is not the handover document. That one tells another developer where things are. This one explains *why they are that way* and *what would break without them*.

PART ONE — ORIENTATION

1 The shape of the whole thing

2 The words you need first

PART TWO — FOLLOW THE DATA

3 Journey: someone opens your site

4 Journey: someone logs in

5 Journey: loading the notes page

6 Journey: buying a premium note

7 Journey: sending a chat message

PART THREE — THE PIECES

8 The frontend, by the problems it solves

9 The backend, and what middleware really is

10 The database, Prisma and migrations

11 Files, and why they are not on your server

PART FOUR — DECISIONS WORTH UNDERSTANDING

12 Nine choices, and what each one costs

PART FIVE — WORKING WITH IT

13 Where to look for what

14 Changing something without breaking it

15 Glossary

PART ONE — ORIENTATION

1. The shape of the whole thing

Before any detail: StudyHub is not one program. It is three separate things that talk to each other over the internet, and almost every confusion about how it works comes from forgetting that.

The three parts

PART	WHERE IT RUNS	WHAT IT IS RESPONSIBLE FOR
Frontend	In the visitor's browser, on their phone or laptop	Everything the user sees and touches. Buttons, pages, forms, the chat window. It holds *no* authority — it asks the backend for everything and displays whatever comes back.
Backend	On a server at Render, somewhere in Frankfurt	All the decisions. Who is allowed to do what, what things cost, whether a payment succeeded. This is the only part you can trust.
Database	On a server at Neon	Remembering things. Every user, note, message, booking and transaction. It only stores and retrieves; it makes no decisions.

> THE ONE IDEA THAT EXPLAINS MOST OF THE DESIGN
>
> **The frontend cannot be trusted.** It runs on someone else's computer. A determined user can open the browser's developer tools, change any value, and send whatever they like to your backend.
>
> So every rule that matters is enforced on the backend. The frontend hides the "Delete" button from a non-owner as a *courtesy*; the backend refuses the delete as the actual *rule*. If you only remember one thing from this document, make it this one.

Why they are separate

You could have built this as one program that generates HTML on the server. Many sites work that way. You did not, for two reasons:

- **Different hosting needs.** The frontend is just files — HTML, JavaScript, images. Files are best served from a CDN: a network of servers worldwide that each keep a copy, so a visitor is served from somewhere near them. The backend cannot work that way: it holds live chat connections open and runs a background timer, which needs one long-lived process.

- **The chat.** Real-time messaging needs a connection that stays open. That is a fundamentally different thing from "send a page, forget the visitor".

The cost of separating them is that they live at different addresses, and browsers block a page at one address from calling another by default. That rule is called CORS, and you had to configure it explicitly. We come back to that in section 9.

How a request actually travels

```
// A student taps "My Notes" on their phone in Buea

 Phone browser
      |  https://studyhubwebapp.vercel.app        1. loads the app (once)
      v
   Vercel CDN  ---> sends back HTML + JavaScript
      |
      |  the JavaScript now runs in the browser and asks for data:
      |  GET https://studyhubwebapp-1.onrender.com/api/notes
      v
   Render (your Express backend)
      |  checks the token, decides what this user may see
      v
   Neon (PostgreSQL)  ---> rows come back
      |
      v
   Render turns rows into JSON ---> back to the phone ---> React draws the list
```

Two separate round trips: one for the app itself, then one for each piece of data it needs.

> **THIS EXPLAINS A BUG YOU ALREADY HIT**
>
> Early on, your deployed site loaded perfectly but every action failed. That is this diagram going wrong at step two: the app itself came from Vercel fine, but it had been built with the wrong backend address, so every data request went nowhere. The page looked healthy because the page *was* healthy — only the second hop was broken.

PART ONE — ORIENTATION

2. The words you need first

Everything after this uses these terms. Read once, refer back as needed. A fuller glossary is in section 15.

TERM	WHAT IT ACTUALLY MEANS, IN YOUR SYSTEM
API	The list of things your backend will do if asked. Each one is a URL plus a method: `POST /api/auth/login`, `GET /api/notes`. You have 88 of these. "Calling the API" just means the frontend sending one of those requests.
Endpoint	One entry in that list. `GET /api/notes/:id` is an endpoint; the `:id` is a slot the caller fills in.
Request / response	One exchange. The frontend sends a request (method, URL, headers, maybe a body of JSON). The backend sends back a response (a status number and usually JSON).
Status code	A number saying how it went. `200` fine. `401` you are not logged in. `403` you are logged in but not allowed. `404` no such thing. `500` the server broke.
JSON	The text format both sides use to exchange data. Looks like a JavaScript object: `{"id": 7, "title": "Calculus notes"}`.
Token	A signed string proving who you are, issued at login. The frontend attaches it to every later request. Covered properly in section 4.
Middleware	A function that runs *before* your actual handler and can stop the request. Your authentication check is middleware. Section 9 explains it fully — it is the single most important backend concept to grasp.
Component	On the frontend: a reusable piece of interface, written as a function that returns markup. `Button` is a component. So is a whole page.
State	Data a component remembers between redraws — what you typed, whether a modal is open, the notes that came back from the API. When state changes, React redraws.
ORM	A translator between your code and the database, so you write `prisma.notes.findMany(...)` instead of SQL. Yours is Prisma. Section 10.
Migration	A recorded, numbered change to the database structure. You have six. They are how the database shape is version-controlled alongside the code.
Environment variable	A setting supplied from outside the code — database passwords, API keys. Kept out of the code so secrets never reach GitHub.
CDN	A worldwide network of servers that each cache a copy of your files, so visitors are served from nearby. Vercel is one; Cloudinary gives you another for uploads.

TERM	WHAT IT ACTUALLY MEANS, IN YOUR SYSTEM
Idempotent	Safe to do twice. Granting a purchase is idempotent in your system: if two things both try to grant it, the student still ends up with exactly one copy of the access, not two. Critical in section 6.

TWO THAT PEOPLE MIX UP CONSTANTLY

401 vs 403. 401 means "I do not know who you are" — log in. 403 means "I know exactly who you are, and you may not do this." Your admin routes return 403 to a logged-in student, not 401.

Authentication vs authorisation. Authentication is *who are you* (the token check). Authorisation is *what may you do* (the role check). Your code separates them into two different middlewares for exactly this reason.

PART TWO — FOLLOW THE DATA

3. Journey: someone opens your site

The simplest journey, and it already contains three things that surprise people.

1. **The browser asks Vercel for the page**

   Vercel sends back a nearly empty `index.html` plus a bundle of JavaScript. The page at this instant is blank. There is no content in the HTML at all.

2. **The JavaScript runs and draws the page**

   React takes over, reads the URL, decides which page component matches, and builds the interface in the browser. This is what "single-page application" means: the server sends one shell, and JavaScript draws everything inside it.

3. **The app checks whether you are already logged in**

   `AuthContext` looks in `localStorage` for a saved token. If there is one, it calls `GET /api/auth/me` to ask the backend whether it is still valid, and only then shows you as signed in.

4. **The page requests whatever data it needs**

   The notes page asks for notes, the dashboard asks for statistics. Each is a separate API call. This is why you sometimes see the layout appear before the content fills in.

The surprising parts

> **WHY DOES IT CHECK THE TOKEN WITH THE SERVER?**
>
> It would be faster to trust the saved token and show the interface immediately. Your code does not, and it is right not to: the token might have expired, or the account might have been suspended since. Asking `/api/auth/me` means a banned user cannot keep using the app just because their browser still holds an old token.
>
> You can see this in `AuthContext.jsx` — if the check fails, it calls `logout()` and clears everything.

THE URL REWRITE THAT MAKES REFRESH WORK

Your routes like `/notes/42` only exist inside the JavaScript. The server has no file at that path. So if someone refreshes the page there, the server would normally return 404.

`vercel.json` fixes this with a rewrite: *every* path returns `index.html`. The browser loads the app, React reads the URL, and draws the right page. Without those four lines in `vercel.json`, every refresh on any page but the home page would 404.

WHAT HAPPENS IF THE BACKEND IS ASLEEP

Render's free tier shuts the backend down after about fifteen minutes of no traffic. The first request afterwards is usually lost while the container starts, which takes 30–50 seconds.

Without handling, a visitor from a shared link sees a broken site. Your `api/client.js` handles it: if a request gets *no response at all*, it waits four seconds and tries again, up to twice, and shows a "waking the server" banner meanwhile.

Note the precision: only requests that got *no response* are retried. If the server answered — even with an error — it is not retried, because repeating a request the server already processed could, for example, charge someone twice.

PART TWO — FOLLOW THE DATA

4. Journey: someone logs in

This is the journey worth understanding in most detail, because authentication is where most real-world security mistakes live — and because two decisions in your code here are subtle enough that an examiner may ask about them.

What happens, step by step

1 **The form sends email and password**

`POST /api/auth/login` with a JSON body. Over HTTPS, so it is encrypted in transit — nobody on the network sees the password.

2 **The backend looks up the user and checks the password**

Here is the actual code, and the comment in it is the interesting part:

```
// backend/src/routes/auth.js
const user = await prisma.users.findUnique({ where: { email } });

// Always run bcrypt compare to prevent timing attacks
const dummyHash = '$2a$12$invalidhashfortimingprotectiononly...';
const isValidPassword = user
    ? await bcrypt.compare(password, user.password)
    : await bcrypt.compare(password, dummyHash).then(() => false);

if (!user || !isValidPassword) {
    return res.status(401).json({ error: 'Invalid credentials' });
}
```

> **WHY COMPARE A PASSWORD FOR A USER THAT DOES NOT EXIST?**
>
> Because bcrypt is deliberately slow — roughly 200–300 milliseconds. If you skipped the comparison when the email was not found, that response would come back almost instantly, while a wrong password for a *real* account would take 300ms.
>
> An attacker can measure that difference. They submit a list of email addresses with any password, and the fast responses tell them which accounts do not exist — which means the slow ones *do*. That is an account enumeration attack, and it is the first step toward targeting real users.
>
> Running the comparison anyway, against a throwaway hash, makes both cases take the same time. The result is discarded. You are burning 300ms on purpose to leak nothing.

3 The backend issues a token

If the password is right and the account is active, it creates a JWT — a JSON Web Token.

```
const token = jwt.sign(
    { userId: user.id, email: user.email },   // the contents
    JWT_SECRET,                               // the signing key
    { expiresIn: '24h' }                      // how long it lasts
);
```

What a token actually is

A JWT is three pieces of text joined by dots. The middle piece is just your data, encoded — **not encrypted**. Anyone holding the token can read the user id and email inside it. Paste one into jwt.io and you will see.

What makes it trustworthy is the third piece: a signature, computed from the contents plus your `JWT_SECRET`. Change a single character of the contents and the signature no longer matches. Since only your server knows the secret, only your server can produce a valid signature.

> **TWO CONSEQUENCES THAT FOLLOW DIRECTLY**
>
> **Never put a secret in a token.** It is readable. Yours holds only a user id and email, which is correct.
>
> **Your server stores nothing.** There is no session table. The token itself carries the proof. This is why the backend could run on ten machines without them needing to share anything — and also why you cannot cancel a token before it expires. Section 12 covers that trade-off.

4 The frontend saves it and attaches it to everything afterwards

```
// frontend/src/context/AuthContext.jsx
function login(userData, token) {
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(userData));
    api.defaults.headers.common['Authorization'] = `Bearer ${token}`;
    setUser(userData);
}
```

`localStorage` is browser storage that survives closing the tab, which is why you stay logged in. The last line tells Axios to attach the token to every future request automatically, so no individual API call has to remember to do it.

5 Every later request is checked by one middleware

This is the whole of `middleware/auth.js`, and it guards most of your 88 endpoints:

```
function authenticate(req, res, next) {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
        return res.status(401).json({ error: 'Authentication required' });
    }
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  // throws if tampered

    req.userId = decoded.userId;    // hand the identity to the real handler
    next();                          // allow the request to continue
}
```

Note `next()`. That is the mechanism: call it and the request proceeds to the actual route handler; return a response instead and the request stops here. Every protected route in your system passes through these twenty lines.

> **THE PATTERN TO RECOGNISE**
>
> `req.userId` is set here and read everywhere downstream. When a route needs to know "who is asking", it never trusts a user id sent in the request body — it uses `req.userId`, which came from a cryptographically verified token. That single discipline prevents a whole category of attacks where someone claims to be another user.

PART TWO — FOLLOW THE DATA

5. Journey: loading the notes page

A routine authenticated read. Worth tracing because it shows the standard shape every one of your routes follows — once you can read this, you can read most of the backend.

The route, in full

```
// backend/src/routes/notes.js
router.get('/', optionalAuth, async (req, res) => {
  try {
    const { subject, group_id, uploaded_by } = req.query;   // 1. read filters

    const where = { is_active: true };                       // 2. build the query
    if (subject)     where.subject     = subject;
    if (group_id)    where.group_id    = parseInt(group_id);
    if (uploaded_by) where.uploaded_by = parseInt(uploaded_by);

    const notes = await prisma.notes.findMany({              // 3. ask the database
      where,
      include: { users: { select: { id: true, first_name: true, last_name: true } } },
      orderBy: { created_at: 'desc' }
    });

    const presented = await Promise.all(                     // 4. decide what to reveal
      notes.map(async n => presentNote(n, await canAccessNote(req.userId, n)))
    );

    res.json(presented);                                     // 5. send JSON back
  } catch (error) {
    console.error('Get notes error:', error);
    res.status(500).json({ error: 'Failed to fetch notes' });
  }
});
```

Reading it line by line

PART	WHAT IT DOES AND WHY
`optionalAuth`	Middleware that sets `req.userId` *if* a valid token is present, but does not reject when there is none. Used because this page is viewable logged-out, yet renders differently when signed in — it can mark which paid notes you already own. The stricter `authenticate` would have blocked visitors entirely.

PART	WHAT IT DOES AND WHY
`req.query`	The bits after `?` in the URL. `/api/notes?subject=Mathematics` arrives as `{ subject: 'Mathematics' }`.
`where`	Built up conditionally, so one endpoint serves all filter combinations. `is_active: true` is always applied — deleted notes are flagged, not removed, and this is what keeps them hidden.
`include`	Fetches the uploader's row alongside each note, in one query. Without it you would fetch notes, then loop and fetch each author separately — the classic "N+1 query" performance mistake.
`select`	Inside the include: takes only id, first and last name. Deliberate. Without it Prisma returns the *whole* user row, including the password hash and email, and you would be publishing them in the API response.
`presentNote`	Strips the file path out of any note the viewer has not paid for, so the response never contains a link they could not legitimately use.
`try / catch`	Logs the real error on the server, returns a generic message to the client. Never send a stack trace to a browser — it tells an attacker about your internals.

THE SHAPE EVERY ROUTE FOLLOWS

Middleware guards → read the input → build the query → ask Prisma → shape the output → respond. Then `try/catch` around all of it. Open any of your twelve route files and you will find the same skeleton. Learning it once is learning all of them.

WHAT BREAKS IF YOU DROP THE `SELECT`

A one-word omission turns this endpoint into a data leak: every note in the list would carry its uploader's full record, hashed password included. The hash is not directly reversible, but publishing it lets an attacker attempt offline cracking at their leisure. This is the kind of mistake that is invisible in the browser — the page looks identical — and obvious in the network tab.

PART TWO — FOLLOW THE DATA

6. Journey: buying a premium note

The hardest journey in the system, and the one you should understand best. Everything difficult about it comes from one fact: **mobile money does not answer immediately, and sometimes never answers at all.**

Why this is harder than a card payment

A card payment is roughly synchronous: you send the details, you wait a second or two, you get a yes or no. Mobile money is not. The sequence is:

1. You tell the provider to collect money from a phone number.
2. The provider returns immediately — saying only "I have accepted the instruction", not "I got the money".
3. The operator pushes a prompt to the payer's handset.
4. The payer types their PIN. Or does not. Or does, eight minutes later. Or their battery dies.
5. At some point the transaction reaches a final state. You only find out by asking.

So the question the design has to answer is: *who is responsible for finding out what happened?* If the answer is "the browser", the system is broken for anyone whose connection is unreliable — which is most of your users.

The flow

1. **Student enters a phone number**

   The frontend detects the operator from the prefix (MTN or Orange) and refuses a number that does not match the selected service, before anything is sent.

2. **The backend creates a transaction and asks the provider**

   Critically, it looks up the price itself. The request from the browser contains *which note*, never *how much*. A transaction row is written with status `pending`, then the collect instruction goes out.

3. **The browser polls**

   Every three seconds for up to three minutes: "is transaction 482 done yet?" The UI shows "check your phone" meanwhile.

4. **On success, access is granted and a receipt issued**

5 **If the browser gives up, the reconciler finishes the job**

A background timer on the server sweeps pending transactions, asks the provider about each, and completes the ones that succeeded — whether or not anyone is still watching.

> **WHY THE RECONCILER IS THE MOST IMPORTANT PART**
>
> Imagine without it. A student approves the payment, then their connection drops, or they close the tab, or they walk out of signal. The money has left their account. Nothing is watching for the confirmation. They have paid and received nothing, and your only recovery is them emailing you.
>
> The reconciler makes this impossible. The browser poll is a convenience that makes the common case feel fast; the reconciler is the actual guarantee. That is why the document says *the browser is an observer, not the authority*.

Two invariants, and how each is enforced

INVARIANT	HOW IT IS ENFORCED
The price cannot be altered	The client never sends an amount. The server reads the price from the `premium_notes` row when it creates the transaction. A tampered request has nothing to tamper with — the field it would change is not consulted.
Access is granted exactly once	The browser poll and the reconciler can both see "success" at the same moment. Granting is done with a *conditional update*: change the status to completed **only if** it is currently pending. The database guarantees only one of the two wins; the loser sees it was already done and stops. This is what "idempotent" means in practice.

Cancellation is not what you would expect

When a student clicks "cancel this payment", the server does not simply mark it cancelled. It asks the provider first, because the payment may have succeeded in the seconds between the student deciding to cancel and the request arriving.

PROVIDER SAYS	SERVER DOES
Success	Completes the order and grants access. They paid; they get the note.
Failed	Marks it failed.
No reference yet	Cancels — nothing was ever charged.
Still pending	Leaves it pending, hands it to the reconciler, tells the user it is still confirming.

THE BUG THAT COST THE MOST TIME, AND WHY

The provider signs each request with a hash computed over the request body. Any character above U+007F — anything outside plain ASCII — makes the signature invalid, and it reports this as **"Bad signature"**.

That message points at your credentials. The credentials were fine. A single typographic dash (—) in one note's title was putting a non-ASCII character into the signed body. Every purchase of *that* note failed; purchases of other notes worked. So the fault looked random and unrelated to content.

The fix is a sanitiser in `mobileMoney.js` that folds typographic punctuation to ASCII, strips anything remaining above U+007F, and truncates. **Any new field you ever send to the provider must pass through it.**

The wider lesson: an error message naming one subsystem is evidence, not proof. I diagnosed the credentials first and was wrong.

PART TWO — FOLLOW THE DATA

7. Journey: sending a chat message

Everything so far used the same pattern: the browser asks, the server answers, the connection closes. Chat cannot work that way, and this section explains why and what replaces it.

The problem with normal requests

HTTP is one-directional. The browser asks; the server answers. The server has no way to say "something happened" on its own — it can only reply to a question.

So how does a message from another person reach your screen? Two options:

APPROACH	HOW IT WORKS	COST
Polling	The browser asks "anything new?" every few seconds, forever.	Wasteful — almost every request returns nothing — and messages are delayed by up to the interval. On metered mobile data this costs the user real money.
WebSocket	One connection opened once and held open. Either side can send at any time.	The server must keep the connection alive, which is why the backend cannot be a serverless function.

Your system uses WebSocket, through a library called Socket.IO, and keeps polling as a slow fallback.

Rooms

With a hundred people connected, a message to group 7 must reach only group 7's members. Socket.IO solves this with *rooms*: each connection joins `group:7`, and the server broadcasts to the room rather than to everyone.

Authenticating a socket

A WebSocket has no `Authorization` header on every message — there is only one connection. So the token is checked once, during the handshake:

```
// backend/src/socket.js
io.use(async (socket, next) => {
    const token = socket.handshake.auth.token;
    if (!token) return next(new Error('Authentication token required'));

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // ... identity attached to the socket, used for every later event
});
```

Same secret, same verification as the REST API — one notion of identity across both channels. And when a socket asks to join `group:7`, the server checks membership *in the database*. It does not take the client's word for which rooms it may enter.

What happens when you hit send

1. **The message appears instantly on your own screen**

   Before the server has heard anything. This is an *optimistic update*: assume it will work, show it immediately, mark it pending. It makes the app feel as fast as WhatsApp.

2. **The message is saved and broadcast**

   The server writes it to `group_messages`, then emits `message:new` to everyone in the room.

3. **Other members' screens update**

   Their clients are listening for that event and add the message. No refresh, no polling delay.

4. **Your own copy is reconciled**

   Your client receives the broadcast too — of a message it already drew. It recognises its own pending message and replaces it rather than adding a duplicate.

WHY YOU WOULD OTHERWISE SEE YOUR OWN MESSAGES TWICE

Step 1 draws it; step 4 broadcasts it back. Without the de-duplication in step 4, every message you sent would appear twice on your screen and once on everyone else's — a bug that only the sender ever sees, which makes it easy to miss in testing.

WHY POLLING IS STILL THERE

Sockets drop, often without a clean disconnect, especially on mobile networks. Your client polls every 30 seconds while the socket is connected, and every 4 seconds when it is not. The slow poll exists to repair events missed during a brief drop — it heals gaps, it does not drive the chat.

WHY DELETED MESSAGES LEAVE A TRACE

Deleting removes the text but keeps the row, flagged as deleted, and the interface shows "This message was deleted". If the row were actually removed, any reply to it would suddenly be answering nothing, and a conversation others had read would silently change meaning. Migration 0003 added this deliberately.

PART THREE — THE PIECES

8. The frontend, by the problems it solves

Fourteen libraries sounds like a lot. Each is there for a specific problem; here is the problem each one solves, so you can justify any of them if asked.

LIBRARY	THE PROBLEM WITHOUT IT	WHAT IT GIVES YOU
React	Updating the page by hand means finding elements and editing them, and keeping that consistent as the app grows is where bugs breed.	You describe what the page *should* look like for a given state; React works out the minimal changes. You never touch the DOM directly.
React Router	A single-page app has no server-side pages, so nothing maps a URL to a screen. Back button and refresh break.	Maps URLs to components. `/notes/42` renders `NoteDetail` with id 42. Back, forward and bookmarks work as expected.
Axios	Every API call would repeat the same token header, timeout, and error handling.	One configured instance in `api/client.js`. Token attached automatically, 45-second timeout, cold-start retry, 401 handling — all in one place.
Tailwind	A stylesheet that grows forever, where nobody dares delete a rule because they cannot tell what uses it.	Styling written in the markup as utility classes. Deleting a component deletes its styling with it.
Framer Motion	Animation in raw CSS gets awkward for things entering and leaving, especially lists.	Declarative animation that handles elements being removed, which plain CSS cannot do well.
Socket.IO client	Raw WebSocket has no reconnection, no rooms, no fallback.	All three, plus automatic reconnection after a drop.
i18next	Two languages means either two copies of every page, or hand-rolled lookup with no plural handling.	One set of pages, text looked up by key. Correct plural rules per language.
react-pdf	A student must download a note before knowing whether it is useful.	Renders PDFs in the page.
Sonner	Feedback via `alert()`, which blocks the page and looks like 1998.	Toast notifications.

LIBRARY	THE PROBLEM WITHOUT IT	WHAT IT GIVES YOU
Recharts	Drawing a chart by hand in SVG.	The dashboard activity chart.
Vite	Browsers want one optimised bundle; you want many small files while developing.	Instant updates while coding; an optimised bundle for production.
vite-plugin-pwa	No offline capability, not installable to a home screen.	Generates the service worker and manifest that make it installable and partly offline.

The two ideas that matter most

State, and why the page redraws

A component holds state — what you typed, whether a modal is open, the notes that came back. When state changes, React redraws that component. You never write "find this element and change its text"; you change the state and let React work it out.

Context, for the few things everything needs

Most state belongs to one component. Two things do not: *who is logged in* and *light or dark theme*. Passing those down through every layer would be miserable, so they live in a Context — a value any component can reach directly. That is all `AuthContext` and `ThemeContext` are.

> **WHY THERE IS NO REDUX**
>
> If asked why you did not use a state-management library: because you did not need one. Two contexts cover the genuinely global state; everything else is local to a component or fetched when a page opens. Adding Redux would have been ceremony without benefit.

> **THE DESIGN SYSTEM, AND THE THEME SWITCH**
>
> Seventeen components in `components/ui/` — Button, Modal, Badge, Input and so on — exist because the same button had previously been written separately on each page and had drifted apart.
>
> Colours are CSS custom properties, and Tailwind's `@theme inline` keeps the generated utility classes pointing at those variables rather than baking in a fixed colour. That is precisely why flipping one attribute on the document root reskins the entire app instantly, with no rebuild.

PART THREE — THE PIECES

9. The backend, and what middleware really is

If you understand middleware, you understand the backend. It is one idea, applied about ten times in a row.

Middleware is a chain of gates

A request arriving at your server does not go straight to the code that answers it. It passes through a line of functions, each of which may inspect it, change it, or stop it. Each gate either calls `next()` — pass it along — or sends a response itself, ending the request there.

```
Request arrives
  |
  v  trust proxy ......... work out the real client IP behind Render's proxy
  v  HTTPS redirect ...... plain HTTP? send a 308 to the https:// address
  v  helmet .............. attach security headers
  v  CORS ................ is this origin allowed to call us? no -> stop, 403
  v  body parser ......... turn the JSON body into a JavaScript object (max 1 MB)
  v  rate limiter ........ too many requests from this IP? stop, 429
  v  authenticate ........ valid token? no -> stop, 401.  yes -> set req.userId
  v  requireRole('admin')  right role? no -> stop, 403
  |
  v  YOUR ROUTE HANDLER ... finally runs; queries the database, replies
  |
  v  error handler ....... anything threw? log it fully, reply with something generic
```

Order is not cosmetic. `authenticate` must run before `requireRole`, because the role check needs to know who the user is. The body parser must run before any handler that reads `req.body`, or the body is not there yet.

What each gate is actually for

GATE	WHY IT EXISTS
trust proxy	Render terminates HTTPS at its own proxy and forwards plain HTTP to you. Without this setting your server thinks every request is insecure and sees the proxy's IP instead of the visitor's — which would make rate limiting count all users as one person.
HTTPS redirect	Sends plain HTTP to the secure address. Uses status **308**, not 301, because 308 preserves the method and body — a redirected POST stays a POST with its data intact.

GATE	WHY IT EXISTS
helmet	Sets a dozen security headers: do not guess file types, do not allow framing, only load scripts from this origin, always use HTTPS next time.
CORS	Browsers block a page on one origin from calling another unless the other explicitly permits it. Your frontend and backend are on different origins, so this permission is mandatory, not optional.
rate limiter	Caps requests per IP. Login is capped at 10 per 15 minutes, which makes password guessing impractical. Payment initiation has its own tighter cap.
authenticate	The token check from section 4.
error handler	Catches anything that threw. Logs the real error with its stack on the server; returns a generic message to the client. A stack trace in a response tells an attacker your file paths and library versions.

> **CORS IS THE ONE THAT WILL BITE YOU AGAIN**
>
> A CORS failure presents as "the frontend is broken" — every request fails, with an opaque browser error and nothing useful in the response. The cause is almost always a trivial mismatch: a trailing slash, `http` where it should be `https`, or a new deployment URL not added to the list.
>
> Because it is so hard to trace, the code has two deliberate aids. A rejected origin is logged *by name* alongside the allow-list, so you can see the mismatch. And if a production server starts with only `localhost` configured, it prints a loud error at startup rather than silently rejecting every visitor.
>
> **If the deployed site suddenly fails on everything, check `ALLOWED_ORIGINS` first.**

Routes, services and the reason for the split

Your backend has twelve route files and seven service files. The distinction is worth keeping:

- **Routes** handle the web layer — read the request, call something, shape the response. They should stay thin.
- **Services** hold the real logic — pricing and entitlement, the payment adapter, the reconciler, file access, the quality gate.

The reason is duplication. Both `payments.js` and `premium.js` take money, and if each contained its own pricing logic they would eventually disagree — and a disagreement about money is the worst kind. Both call `entitlements.js` instead, so there is exactly one implementation of "what does this cost and who may now have it".

A GOOD RULE WHEN ADDING FEATURES

If a route handler grows past about forty lines, or if you find yourself copying logic into a second route, that logic wants to be a service.

PART THREE — THE PIECES

10. The database, Prisma and migrations

Twenty-five tables, and one file that defines all of them.

Why a relational database

Your data is full of relationships: a user belongs to many groups, a group has many members, a note has one uploader, a booking joins a student to a tutor. A relational database enforces those links — it will refuse to store a note whose uploader does not exist.

If asked why not MongoDB: because you would have to enforce every one of those rules in application code, and any path that forgot would quietly corrupt the data. The database doing it means it cannot be forgotten.

What Prisma does

Without an ORM you would write SQL strings, and build them by joining text — which is how SQL injection happens. Prisma gives you three things:

- **One schema file** defining every table, in `prisma/schema.prisma`. The single source of truth.
- **A generated client** so you write `prisma.notes.findMany({ where: { subject: 'Maths' } })` and get back objects. Values are parameterised, so injection is not possible by construction.
- **Migrations** — recorded changes that bring any database to the current shape.

Migrations, and why they matter

Your database has changed six times. Each change is a numbered, committed file. This means your local database, the production database and a teammate's can all be brought to the same shape by running the same sequence — and the shape of your data is version-controlled next to the code that depends on it.

The alternative is editing tables by hand, after which nobody can reproduce the structure and production drifts away from development. **Never alter the deployed database by hand.** Change the schema file, generate a migration, commit it.

```
# after editing schema.prisma
npx prisma migrate dev --name add_something   # local: create + apply + regenerate
npx prisma migrate deploy                     # production: apply only, never create
npx prisma studio                             # browse the data in a GUI
```

The two connection strings

This trips people up, so it is worth understanding rather than memorising.

VARIABLE	WHICH ENDPOINT	USED FOR, AND WHY
`DATABASE_URL`	Pooled (`-pooler` in the hostname)	The running app. A pooler shares a small set of real connections among many requests, which keeps you inside the free tier's connection limit.
`DIRECT_URL`	Direct	Migrations only. The pooler works in transaction mode and cannot carry session-level operations — and migrations need exactly those: advisory locks and type creation.

THE FAILURE THIS CAUSES

Run a migration through the pooled URL and it fails with a message that says nothing about pooling. You can lose an hour to it. The giveaway is `-pooler` in the hostname: present means the app's URL, absent means the migrations' URL.

Two modelling decisions worth being able to defend

VOTES ARE ROWS, NOT A COUNTER

A `votes` integer on each question would be simpler. Instead each vote is its own row in `question_votes`, with a uniqueness constraint on (voter, question).

Why: with a counter, nothing stops the same person voting fifty times except application code remembering to check — and any new code path that forgets defeats it. With a constraint, the database physically refuses the second vote. The guarantee sits where it cannot be bypassed.

ENTITLEMENT IS A ROW, NOT A FLAG

Access to a paid note is "does a `purchased_notes` row exist for this user and note", not a boolean somewhere. That also gives you an audit trail: who bought what, when, and under which transaction. A flag would have told you only that *someone* had access, with no way to reconstruct why.

PART THREE — THE PIECES

11. Files, and why they are not on your server

Every avatar, note, chat attachment and voice recording goes to Cloudinary rather than to disk. There is a hard reason and a useful one.

The hard reason: the disk disappears

Render gives your app an *ephemeral* filesystem. Every time you deploy, the container is rebuilt from scratch and anything written to disk is gone. Had you stored uploads locally, every deployment would have destroyed every file students had uploaded since the last one.

This is not a Render quirk — it is normal for modern hosting. Application servers are treated as disposable. Anything that must survive goes to a database or an object store.

The useful reason: delivery

Cloudinary is also a CDN. A student in Buea downloading a note gets it from a nearby edge server rather than from Frankfurt, and it never passes through your single Node process.

Two delivery paths

CONTENT	HOW IT REACHES THE USER
Free	The API responds with a **redirect** to the Cloudinary URL. Your server says "go and get it there" and steps out of the way entirely.
Paid	Uploaded under Cloudinary's *authenticated* delivery type, which cannot be fetched from a plain URL at all. The server checks for a purchase row, and only then generates a short-lived **signed URL**.

> **THE ONE PERFORMANCE MEASUREMENT IN THE PROJECT**
>
> Before the redirect, a 3.19 MB PDF streamed through the API did not finish within the 300-second client timeout. After, the same file on the same connection arrived in 31 seconds.
>
> Be precise about this if you quote it: 300 seconds is a *timeout*, not a completion. The honest claim is "undeliverable before, 31 seconds after" — not a tenfold speedup of something that previously worked. The gain comes from removing a constrained single process from the data path.

WHAT A SIGNED URL IS

A normal URL works for anyone who has it, forever. A signed URL carries a cryptographic signature and an expiry, both checked by Cloudinary. Alter any part of it and it stops working; wait past the expiry and it stops working.

So the *decision* (has this person paid?) is made by your server, while the *delivery* is done by Cloudinary. You get the access control without the bandwidth.

Be careful not to overclaim this in a defence. It prevents access *without purchase*. It does not prevent a legitimate buyer from sharing the file afterwards — no system can.

PART FOUR — DECISIONS

12. Nine choices, and what each one costs

Every engineering decision buys something and costs something. If you can name both sides for each of these, you can defend the system.

DECISION	WHAT IT BUYS	WHAT IT COSTS
Hash passwords with bcrypt, never encrypt	Encryption is reversible; hashing is not. If the database leaks, nobody can recover the passwords. bcrypt is also deliberately slow, so guessing at scale is impractical.	About 300ms per login, intentionally. You cannot ever show a user their password — only reset it.
Stateless JWT instead of server sessions	No session storage. The API could run on many machines with nothing shared. The WebSocket handshake reuses the same check.	**You cannot revoke a token.** Sign out or ban an account and the existing token still works until it expires. The 24-hour expiry bounds the damage but does not remove it. This is a genuine weakness — name it before an examiner does.
Re-read the role from the database	A demotion takes effect immediately rather than at token expiry.	One extra query per protected admin request. Worth it.
Resolve prices server-side	A tampered request cannot change what is charged, because the field it would alter is never read.	None worth mentioning. This is simply correct.
Background reconciler	A successful charge always grants access, even if the payer disconnects. The guarantee behind the whole payment flow.	A background process, and transactions can sit pending for a while before resolving.
Redirect free downloads to the CDN	Large files become deliverable at all, and your server stays free to handle requests.	The file URL is public — which is fine, because the content is free.
Soft-delete messages	Conversations stay coherent; replies do not end up answering nothing.	Rows are never truly removed, so the table only grows.
Compute uploader trust, never store it	Trust lapses by itself when the recent record no longer supports it. Nothing has to remember to revoke a flag.	A small query each time instead of reading one column.

DECISION	WHAT IT BUYS	WHAT IT COSTS
Translate labels, never stored values	Subjects and categories stay comparable across languages, so French and English users find each other's content.	Every display of a stored value needs a lookup with a fallback. The failure mode if you forget is *silent* — no error, just empty search results.

PART FIVE — WORKING WITH IT

13. Where to look for what

A map for when you need to change something and do not know where it lives.

I WANT TO CHANGE…	LOOK IN
What a page looks like	`frontend/src/pages/` — one file per screen
A button, modal, badge — anywhere it appears	`frontend/src/components/ui/`
Colours, spacing, the dark theme	`frontend/src/index.css` — the design tokens
Any text the user reads	`frontend/src/i18n/locales/en.json` and `fr.json` — **both**
Which URL shows which page	`frontend/src/App.jsx`
How API calls behave (timeout, retry, token)	`frontend/src/api/client.js`
What an endpoint does	`backend/src/routes/` — named after the area
Pricing or granting access	`backend/src/services/entitlements.js`
Anything touching the payment provider	`backend/src/services/mobileMoney.js` — and nowhere else
Who may do what	`backend/src/middleware/`
The shape of the data	`backend/prisma/schema.prisma`
Startup, security headers, CORS, rate limits	`backend/src/index.js`

Reading an unfamiliar route

Open any route file and read in this order: the middleware on the route line (who may call this), then the first few lines of the handler (what input it takes), then the Prisma call (what it touches), then what it sends back. That is usually enough to understand it without reading every line.

PART FIVE — WORKING WITH IT

14. Changing something without breaking it

The checks you have

COMMAND	WHAT IT CATCHES
`npm run build`	**The real gate.** A mistyped design token resolves silently in development and fails here. If the build fails, do not deploy.
`npm run lint`	Undefined variables, unused imports, bad hook usage. The baseline is 42 errors and 13 warnings, all pre-existing — your change should not increase either number.
`npm run check:i18n`	A key missing from one language, mismatched placeholders, an incomplete plural, a key used but never defined. Run it after any text change.
`npx prisma studio`	Not a check, but the fastest way to see what actually landed in the database.

> **THE CHECK YOU DO NOT HAVE**
>
> There is no automated test suite. Nothing verifies that the reconciler still grants exactly once, or that a student cannot reach a note they did not buy. Those are currently guaranteed by code review and by you testing manually.
>
> This is the project's biggest weakness and you should say so if asked. The evidence is in your own defect record: every fault caught before deployment was caught by a mechanical check, and every fault that reached users was in an area with no check.

A safe routine for a change

1. **Make the change**

   If it touches text, update both `en.json` and `fr.json`.

2. **Run the checks**

   `npm run build`, then `lint`, then `check:i18n` if text changed.

3. **Exercise it by hand**

   The actual path a user takes. If it touches the interface, check both languages and both themes, and narrow the window to phone width.

4 **Commit and push**

Both hosts deploy automatically. If the schema changed, run `npx prisma migrate deploy` against the *direct* URL.

5 **Check the deployed site**

Not just that it loads — exercise the thing you changed.

Things that will bite you

SYMPTOM	ALMOST ALWAYS
Everything fails after a deploy	CORS. Check `ALLOWED_ORIGINS` includes the exact origin — scheme, no trailing slash.
Site loads, every request fails	`VITE_API_URL` wrong or changed without a redeploy. It is baked in at *build* time.
First visit after a quiet spell is slow	Render cold start. Expected; the banner explains it to the user.
Payment says "Bad signature"	Either wrong keys, or a non-ASCII character reached the provider. See section 6.
`EPERM` on `prisma generate`	The dev server is holding the engine file. Stop the backend first.
Uploads fail with a 500	`CLOUDINARY_*` not set.
French text shows a key name	Missing key. `npm run check:i18n` will name it.

PART FIVE — WORKING WITH IT

15. Glossary

Everything in one place.

TERM	MEANING
API	The set of operations your backend exposes. Yours has 88.
Axios	The library the frontend uses to call the API.
bcrypt	Password hashing designed to be slow, so guessing is impractical. Yours runs at work factor 12.
CDN	Content Delivery Network — servers worldwide caching your files so users are served from nearby.
Component	A reusable piece of interface in React.
Context	React's way of sharing a value with any component without passing it down manually. You have two: auth and theme.
CORS	The browser rule that a page on one origin may not call another unless permitted. Required because your two tiers are on different origins.
Endpoint	One operation in the API: a method plus a path.
Entitlement	The record that someone is allowed something they paid for. A row in `purchased_notes`.
Environment variable	Configuration supplied from outside the code, so secrets stay out of GitHub.
Hashing	One-way transformation. Unlike encryption it cannot be reversed, which is exactly why passwords use it.
Idempotent	Safe to perform more than once with the same end result. Your purchase granting is.
JSON	The text format for data exchange between frontend and backend.
JWT	JSON Web Token. A signed, *readable* token proving identity. Yours lasts 24 hours.
localStorage	Browser storage that survives closing the tab. Holds your token, theme and language.
Middleware	A function running before the route handler that can inspect, modify or stop a request.
Migration	A recorded, numbered change to the database structure. You have six.

TERM	MEANING
Optimistic update	Showing a change immediately, before the server confirms. Used in chat so it feels instant.
ORM	Object-Relational Mapper. Lets you query the database in JavaScript instead of SQL. Yours is Prisma.
Origin	Scheme + domain + port. `https://studyhubwebapp.vercel.app` is one origin.
Polling	Repeatedly asking "anything new?". Your fallback when the socket is down.
Pooled connection	A shared set of database connections reused across requests, to stay within tier limits.
PWA	Progressive Web App — a website installable to a home screen that works partly offline.
Rate limiting	Capping requests per IP per time window, to blunt brute-force and abuse.
Reconciler	Your background sweep that settles payments the browser stopped watching.
Route	Backend: code answering one endpoint. Frontend: a URL mapped to a page.
Service	A backend module holding domain logic, called by routes.
Service worker	Background browser script enabling offline behaviour and caching.
Signed URL	A time-limited, tamper-proof link. Used to release paid files after an entitlement check.
Soft delete	Flagging a row as deleted rather than removing it.
SPA	Single-Page Application. One HTML shell; JavaScript draws every screen.
State	Data a component remembers between redraws.
Timing attack	Deducing secrets from how long a response takes. Your login defends against it deliberately.
Token	See JWT.
WebSocket	A connection held open so either side can send at any time. Used for chat.

IF YOU REMEMBER FIVE THINGS

1. The frontend cannot be trusted; every rule is enforced on the backend.
2. Middleware is a chain of gates, and `next()` is what lets a request through.
3. The browser observes payments; the reconciler is what guarantees them.
4. Stored values stay English; only labels are translated.
5. `npm run build` is the real gate, not the dev server.

Companion to *StudyHub — Technical Documentation* (reference) and *StudyHub — Academic Project Report* (evaluation). This one is the explanation.